

Linguagem de Programação Orientada a Objetos

NOTAS DE AULA

Paulo A. Pagliosa

Capítulo 0

Geração de Programas Executáveis em C/C++

O processo de geração de um arquivo executável em C ou C++ engloba os passos a seguir, ilustrados na Figura 0.1.

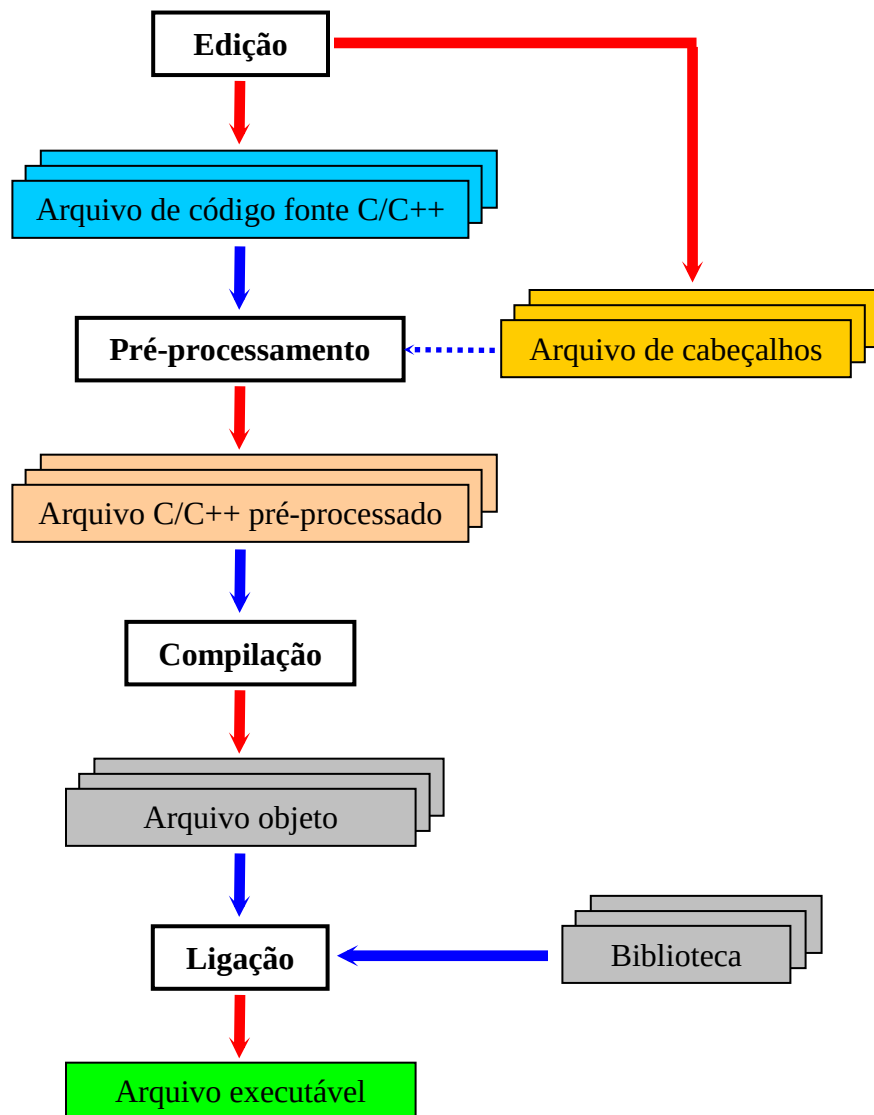


Figura 0.1 Processo de geração de um programa executável em C/C++.

Edição do Código Fonte

A edição do código fonte pode ser feita em qualquer editor de texto, mas preferencialmente em um ambiente de desenvolvimento integrado, ou IDE (*integrated development environment*, em inglês). O editor de textos dos IDEs atuais, como o Visual Studio da Microsoft, por exemplo, vem com destaque de sintaxe (ou *syntax highlighting*, em inglês): à medida que se digita o código, o editor destaca, usando fontes e cores configuráveis, os diferentes elementos léxicos que compõem o programa, tais números inteiros ou reais, strings, palavras reservadas, identificadores, etc., o que torna o código muito mais legível. Embora vários editores de texto contem com este recurso, os IDEs possuem muitos outros, como complementação e formatação de código, gerenciamento de projetos, compilação, execução e depuração embutidas; entre tantos.

Na edição de programas C/C++ produzem-se tipicamente dois tipos de arquivos texto: *arquivos de cabeçalhos* e *arquivos de código fonte* propriamente dito, os últimos contendo o código de funções e/ou declarações de variáveis globais. Um arquivo de cabeçalhos tem extensão `.h` (ou `.hpp` ou `.hxx`, em C++) e contém declarações de variáveis globais externas e/ou protótipos de funções. Um protótipo (ou um cabeçalho) de uma função é a declaração da função, com seu nome, tipo de retorno e número e tipo de parâmetros, mas sem o corpo. Por exemplo, seja uma função chamada `f()` que retorna um inteiro e que toma como argumentos um inteiro e um ponteiro de inteiro. O protótipo da função é

```
int f (int ,int*);
```

O propósito de um arquivo de cabeçalhos é reunir protótipos de funções que poderão ser *invocadas* em um ou mais arquivos de código fonte C/C++. Por exemplo, o arquivo de cabeçalhos `stdio.h` contém os protótipos das funções da biblioteca de entrada e saída padrão do C/C++ (note: `stdio.h` declara os protótipos das funções da biblioteca, mas *não* é a biblioteca, uma vez que não contém o código binário das funções nele declaradas). Os protótipos são necessários porque, na invocação de uma função em um programa, o compilador deve conhecer a priori o tipo de retorno e o número e tipo de cada argumento da função sendo invocada, a fim de verificar a correte semântica do programa. Assim, todo arquivo fonte C/C++ que invocar a função `printf()`, por exemplo, pode incluir o arquivo de cabeçalhos `stdio.h`, a fim de que o compilador conheça o protótipo da função `printf()` e possa compilar o arquivo fonte sem erros.

Arquivos de código fonte C++ (objetos deste curso) têm extensão `.cpp` (ou `.cxx`) e contém o código fonte do corpo de funções e/ou a declaração de variáveis globais. Os protótipos das funções implementadas e/ou invocadas em um arquivo `.cpp` podem ser declaradas em um ou mais arquivos de cabeçalhos `.h`, os quais então devem ser incluídos no arquivo de código-fonte C++. A inclusão de um arquivo de cabeçalho em um arquivo C++ é feita com a diretiva de pré-processamento `#include`

Pré-Processamento

Uma vez editados arquivos `.h` e arquivos `.cpp` (os quais incluem arquivos `.h`), o próximo passo para geração do arquivo executável é o pré-processamento. O pré-processor é um programa de computador que toma como entrada um arquivo de código fonte `.cpp` e gera como saída um arquivo temporário com código fonte pré-processado (o qual será entrada do compilador). As principais tarefas, mas não as únicas, do pré-processor são:

- Inclusão de arquivos. Ao encontrar a diretiva `#include` o pré-processador procura o arquivo cujo nome é dado após a diretiva. Se o arquivo puder ser encontrado, o pré-processador o abre e expande o seu conteúdo no arquivo temporário de saída, no lugar da diretiva `#include`. A expansão é feita recursivamente, ou seja, se o conteúdo a ser expandido contiver outras diretivas de pré-processamento, estas são tratadas pelo pré-processador. O nome do arquivo a ser incluído é dado entre aspas duplas, como uma string, ou entre `<` e `>`. No primeiro caso, o pré-processador busca o arquivo no diretório corrente; no segundo, no caminho (*path*) do sistema.
- Expansão de macro-comandos. Um macro-comando, ou apenas macro, é definido pela diretiva de pré-processamento `#define`, seguida do nome e, opcionalmente, da lista de argumentos do macro (cujo parêntese esquerdo sucede o nome do macro sem espaços) e do corpo do macro. Os macros definidos no código fonte de entrada do pré-processador são suprimidos no arquivo de saída temporário. A expansão de macros se da seguinte forma: o pré-processador varre o texto do código-fonte e, para cada nome encontrado, verifica se este nome é igual ao nome de algum macro definido com a diretiva `#define`. Se for, então o corpo do macro, se definido, é expandido no lugar do nome. Se o macro tiver argumentos, estes também serão expandidos, como no exemplo abaixo.

Código fonte

```
#define DEBUG
#define MAX_SIZE 1024
#define MAX(a,b) ((a)>(b)?(a):(b))

void foo(int v[MAX_SIZE])
{
    for (int i = 0, j = MAX_SIZE; i < MAX_SIZE;)
        v[i] = MAX(i++, j--);
}
```

Código fonte pré-processado

```
void foo(int v[1024])
{
    for (int i = 0, j = 1024; i < 1024;)
        v[i] = ((i++)>(j--)?(i++):(j--));
}
```

- Compilação condicional. Dentre as variantes possíveis de combinações de diretivas para compilação condicional, vamos nos deter nas usadas no exemplo a seguir.

```
#ifdef DEBUG
...
#else
...
#endif
```

A diretiva `#ifdef` verifica se o macro `DEBUG` foi ou não definido. Se foi, todo o código a partir da linha seguinte a diretiva e antes da linha da diretiva `#else` é pré-processado (ou seja, contribui para o conteúdo do arquivo de saída gerado pelo pré-processador). Caso contrário, isto é, se o macro `DEBUG` não foi definido, o código a ser pré-processado é aquele entre as linhas das diretivas `#else` e `#endif`. O uso de

tais diretivas de pré-processamento, portanto, permite a seleção de trechos de código que serão submetidos ao compilador. Outro exemplo com mais diretivas:

```
#ifndef WIN32
#include <dir.h>
#elif defined(LINUX)
extern void _splitpath(const char*, char*, char*, char*);
#else
#error "SO não definido"
#endif
```

Embora sejam passos distintos, na prática, o pré-processamento ocorre em conjunção e durante a compilação.

Compilação

O compilador toma como entrada um arquivo texto (gerado pelo pré-processador) contendo declarações escritas na linguagem C++ e produz como saída um arquivo binário chamado arquivo de código objeto (cuja extensão é `.o` no Linux e `.obj` no Windows). Este contém as instruções em linguagem de máquina de todas as funções e expressões do código C++. Se o código-fonte contiver algum erro léxico, sintático ou semântico, o computador emite uma mensagem correspondente e não gera o arquivo objeto.

Para cada arquivo C++ (sem erros) submetido ao compilador, um arquivo objeto será produzido. Um arquivo objeto ainda não é um arquivo executável, pois contém apenas o código em linguagem de máquina do arquivo fonte C++ que lhe deu origem. Por exemplo, seja um arquivo chamado `a.cpp` com o seguinte conteúdo:

```
#include <stdio.h>

int main()
{
    puts("Prog II");
}
```

No Linux o compilador gerará o arquivo `a.o` contendo, entre outras informações, o código binário da função `main()`. Note que a função `main` invoca a função `puts()`, mas o código do corpo desta função não está definido no arquivo `a.cpp` nem em `stdio.h`. Uma vez que o protótipo de `puts()` é conhecido (pois está declarado em `stdio.h`), o compilador gera código binário com a instrução em linguagem de máquina para *invocar* a função, mas o código binário do corpo de `puts()` propriamente dito não faz parte de `a.o`. Este código está em *outro* arquivo objeto (mais precisamente, está em um arquivo que reúne vários arquivos objetos e que é chamado *biblioteca*) e, portanto, deve ser anexado ou *ligado* ao código binário de `a.o` a fim de que, após esta ligação, tenha-se um programa que possa ser executado pelo SO. Esta tarefa é desempenhada pelo ligador.

Ligação

Um arquivo executável (a extensão é `.exe` no Windows) é gerado por um ou mais arquivos de código objeto passados ao ligador. Além de arquivos objetos, o ligador pode receber um ou mais arquivos de biblioteca (`.lib` no Windows) que nada mais são que uma coleção compacta de um ou mais arquivos objetos. O ligador C++ também toma como entrada automaticamente as bibliotecas e os arquivos objetos padrões do C++. O papel do ligador é

gerar o arquivo executável a partir do código binário de todos os arquivos objetos passados como entrada, *resolvendo* (isto é, determinando o endereço) de todas as invocações de funções nos arquivos objetos. Se o corpo de alguma função invocada em algum arquivo objeto não puder ser encontrado no ou em qualquer outro arquivo objeto, o ligador emite uma mensagem de erro e não gera o arquivo executável.

O compilador C/C++ usado no curso será o `gcc` (`g++` para C++). Este, de fato, não é apenas um compilador, mas um programa que, dependendo dos argumentos passados na linha de comando, executa automaticamente o pré-processador, o compilador propriamente dito e o ligador a fim de produzir, a partir dos arquivos de código fonte C++ (`.cpp`) e objetos (`.o`) dados como entrada, um arquivo executável de saída.

Capítulo 1

Fundamentos da Orientação a Objetos

Um *objeto* é um *modelo* computacional de uma entidade concreta ou abstrata, definido por um conjunto de *atributos* e capaz de executar um conjunto de operações ou *métodos*. Os atributos de um objeto representam os dados que caracterizam a estrutura da entidade; os métodos executam sobre os atributos e definem o comportamento da entidade. O conjunto de atributos e/ou de métodos de um objeto pode ser vazio.

Uma *classe* é uma descrição dos atributos e dos métodos de um determinado tipo de objeto. Definida uma classe pode-se, a partir dela, construir objetos daquele tipo. Um objeto é uma *instância* de uma classe. Em C++, uma classe pode ser sintaticamente definida como uma **struct** na qual se declaram, além dos campos (doravante chamados atributos), também funções, ou, mais propriamente, métodos.

Exemplo 1.1

A classe `IntList` a seguir define um tipo lista de números inteiros.

```
struct IntList // classe
{
    // declaração de atributos
    IntNode* head;
    IntNode* tail;
    int numberOfElements;

    // declaração de métodos
    IntList(); // construtor
    ~IntList(); // destrutor
    void add(int);
    bool remove(int);
    void clear();
    bool contains(int);
    void iterate(void (*) (int));
};
```

Definida a classe `IntList` podemos criar uma instância da classe, ou seja, um objeto do tipo `IntList`:

```
IntList list;
```

A declaração acima cria um objeto chamado `list` do tipo `IntList`. O objeto `list` contém os atributos `head`, `tail` e `numberOfElements`, os quais foram declarados na classe `IntList`. O objeto `list` é capaz de executar os métodos `add()`, `remove()`, `clear()`, `contains()` e `iterate()`, todos declarados na classe `IntList`. Os atributos declarados em `IntList` representam a estrutura do objeto `list`. Os métodos declarados na classe `IntList` representam o comportamento do objeto `list`. O objeto `list` é uma instância da classe `IntList`.

1.1 Ciclo de Vida de um Objeto

O ciclo de vida de um objeto consiste na *criação*, *uso* e *destruição* do objeto.

Criação de Objetos

Em C++, um objeto pode ser criado de duas maneiras: *estaticamente* e *dinamicamente*. Para se criar um objeto de forma estática, basta declarar uma variável cujo tipo é a classe do objeto, como feito anteriormente:

```
IntList list;
```

Na declaração acima, `list` é um objeto da classe `IntList`. A criação dinâmica de um objeto é feita com a utilização do operador **new**:

```
IntList* p = new IntList;
```

Na declaração acima, `p` é um ponteiro para um objeto da classe `IntList`, cujo valor é o endereço de uma instância de `IntList`, dinamicamente criada com o operador **new**.

A criação de um objeto, seja estática ou dinâmica, envolve:

1. Alocação de espaço na memória para armazenamento dos valores dos atributos do objeto, atributos esses declarados na classe do objeto. Um objeto criado estaticamente tem memória reservada em tempo de compilação, no seguimento de dados ou no segmento de pilha do programa, caso o objeto tenha sido declarado global ou localmente (ou seja, em um bloco de função), respectivamente. Um objeto criado dinamicamente tem memória alocada em tempo de execução no heap do programa.
2. Inicialização do objeto. A inicialização usualmente é feita através da atribuição de valores iniciais aos atributos do objeto recém criado, mas pode envolver quaisquer ações que se fizerem necessárias, tais como alocação de memória, abertura de arquivos, envio de mensagens via rede, impressão na tela, etc. O código de inicialização de um objeto é escrito em métodos especiais, declarados na classe do objeto, chamados de *construtores*.

Em C++ um método construtor não possui tipo de retorno (nem mesmo **void**), e tem o nome idêntico ao nome da classe. Uma classe pode declarar vários construtores, desde que distinguidos pelo número e/ou tipo de parâmetros formais.

Um construtor que não possui quaisquer parâmetros é chamado construtor *default*. A classe `IntList` declara um construtor default que inicializa os atributos da lista recém criada tal que esta seja vazia. A implementação do método (efetuada fora da classe) é dada a seguir:

```
IntList::IntList()  
{  
    head = tail = 0;  
    numberOfElements = 0;  
}
```

Observações

- Em C++, a implementação, ou seja, a definição do corpo, de um método declarado em uma classe pode ser efetuada dentro da classe ou fora da classe. Assim, a implementação do construtor default da classe `IntList` poderia ser feita dentro da classe:

```

struct IntList
{
    ...
    IntList() // construtor default
    {
        head = tail = 0;
        numberOfElements = 0;
    }
    ...
};

```

- Um método implementado fora da classe no qual foi declarado deve ser identificado através de seu *nome qualificado*. O nome qualificado de um método `m()` declarado em uma classe `x` é `x::m()`, onde o nome `x` é chamado *qualificador*, o nome `m` é chamado *nome simples* e `::` é o *operador de escopo* de C++.
- Um método implementado dentro de sua classe é *inline* por definição; um método declarado fora da classe não é *inline* a não ser que o modificador **inline** preceda a assinatura do método:

```

inline
IntList::IntList()
{
    head = tail = 0;
    numberOfElements = 0;
}

```

- Uma função ou método `m()` *inline* informa ao compilador para que este tente, na geração de código nas invocações de `m()`, substituir a chamada por uma expansão "em linha" do corpo de `m()`. A invocação de uma função ou método *inline* (caso o compilador consiga fazer a expansão em linha), elimina o *overhead* da passagem de argumentos e saltos de ida e de volta de uma invocação de função ou método não *inline*, mas, em contrapartida, se o corpo for muito grande e houver muitas invocações a `m()`, o código final tende a ficar maior. Portanto, funções ou métodos *inline* devem ser usados com critério.
- Como o corpo de uma função ou método *inline* deve ser conhecido no momento de uma invocação da função ou método, a implementação deste é comumente efetuada em um arquivo de cabeçalho (`.h`).

Exemplo 1.2

Escrever uma classe de números complexos com os atributos e construtores.

```

struct Complex
{
    float a; // parte real
    float b; // parte imaginária

    Complex() // construtor default
    {
    }
    Complex(float aa)
    {
        a = aa;
        b = 0;
    }
}

```



```

Complex(float aa, float ab)
{
    a = aa;
    b = ab;
}
Complex(const Complex& c) // construtor de cópia
{
    a = c.a;
    b = c.b;
}
};

```

Definida a classe podem-se criar números complexos:

```

Complex c1;           // invoca o construtor default (sem argumentos)
Complex c2();         // default, como no caso acima
Complex c3(5);        // Complex::Complex(float)
Complex c4(3, 4);     // Complex::Complex(float, float)
Complex c5(c4);       // de cópia
Complex c6 = c5;      // de cópia, como no caso acima

```

Um construtor *de cópia* em uma classe *x* é caracterizado por ter apenas um parâmetro cujo tipo é uma *referência* para um objeto *constante* da própria classe *x* (o parâmetro é constante, pois seus atributos não são alterados dentro do corpo do construtor). O propósito de um construtor de cópia é inicializar os atributos do objeto sendo criado com uma cópia dos atributos do objeto referenciado pelo parâmetro.

Observações

- Em C++, se *nenhum* construtor for declarado em uma classe, o compilador automaticamente provê um construtor default com um corpo vazio, ou seja, que não faz nada. No caso de *algum* construtor ser declarado e o default ser necessário, o programador deve prover um.
 - Em C++, se o construtor de cópia não for declarado em uma classe, o compilador *sempre* provê um (independente da declaração de outros construtores), o qual faz uma cópia atributo a atributo do parâmetro para o objeto sendo inicializado. Assim, o construtor de cópia da classe `Complex` não precisaria ser explicitamente definido.
-

Uso de Objetos

Seja o objeto criado estaticamente:

```
Complex c(5, 6);
```

Uma das maneiras de se usar o objeto *c* é *acessar* um de seus atributos, seja para obter ou alterar seu valor:

```
float a = c.a;
c.b = 2;
```

Acima, *c.a* é um *d-valor* (valor à direita em uma expressão de atribuição) e *c.b* é um *e-valor* (valor à esquerda). Da mesma forma tem-se, para um objeto criado dinamicamente:

```
Complex* p = new Complex(5, 6);
```

```
float a = p->a;
p->b = 2;
```

Exemplo 1.3

Estender a funcionalidade da classe `Complex` com as operações de adição e subtração de números complexos.

```
struct Complex
{
    float a; // parte real
    float b; // parte imaginária

    Complex() // construtor default
    {
    }
    Complex(float aa)
    {
        a = aa;
        b = 0;
    }
    Complex(float aa, float ab)
    {
        a = aa;
        b = ab;
    }
    Complex(const Complex& c) // construtor de cópia
    {
        a = c.a;
        b = c.b;
    }

    Complex add(const Complex& c) const
    {
        // Ao invés de
        // -----
        // Complex temp(a + c.a, b + c.b);
        // return temp;
        // -----
        // pode-se escrever
        return Complex(a + c.a, b + c.b);
    }
    Complex sub(const Complex& c) const
    {
        return Complex(a - c.a, b - c.b);
    }
};
```

Declarados os métodos que um objeto pode executar, outra maneira de usar um objeto é *enviar* a este uma *mensagem*. Uma mensagem é uma solicitação feita a um objeto para que este execute uma determinada operação. Para exemplificar, seja um objeto `x` da classe `x`, criado estaticamente. O envio de uma mensagem a `x` tem a seguinte sintaxe em C++:

```
x.m(1, 2, 3);
```

onde o objeto x é chamado *receptor* da mensagem, m é o *seletor* da mensagem e $(1, 2, 3)$ é a lista de *argumentos* da mensagem. Em resposta a uma mensagem, o objeto deve executar um método. No exemplo acima, o método a ser executado será aquele declarado na classe x cujo nome simples é m e cujo número e tipo de parâmetros são compatíveis com o número e tipo dos argumentos da mensagem (neste caso três números inteiros). O mecanismo de seleção de um método, em resposta a uma mensagem enviada a um objeto, é chamado *acoplamento mensagem/método*.

Exemplo 1.4

```
Complex c1(1, 2);  
Complex c2(3, 4);  
Complex c3 = c1.add(c2); // construtor de cópia
```

O argumento do construtor de cópia usado na inicialização do objeto $c3$ é resultado da mensagem

```
c1.add(c2);
```

onde $c1$ é o receptor, add é o seletor e a referência ao objeto $c2$ é o argumento. O método acoplado à mensagem é `Complex::add()`, pois este é o método declarado na classe (`Complex`) do receptor ($c1$) que possui nome simples (add) idêntico ao seletor da mensagem e número (1) e tipo (`const Complex&`) compatíveis com o número e tipo de argumentos ($c2$) da mensagem.

Em programação orientada a objetos (POO) a computação ocorre em um universo constituído de objetos que trocam mensagens entre si. Dentro desse universo, objetos são criados, possuem um tempo de vida útil e são destruídos ao longo da computação.

Observações

- Pode-se enviar a um objeto somente mensagens para as quais há métodos que, declarados na classe do objeto, podem ser acoplados às mensagens.
- Sintaticamente, o envio de uma mensagem é similar ao acesso a um atributo, com a diferença que há um par de parênteses com ou sem argumentos. Ou seja, em termos sintáticos, pode-se pensar em uma mensagem como sendo uma espécie de "acesso a um método" ou "chamada de função" do objeto. Semanticamente, porém, esta analogia não vale. O que vale é que um método da classe será executado como resultado do acoplamento mensagem/método.

Sobrecarga de operadores

Em C++, o nome de um método pode ser baseado em um operador da linguagem (a maioria dos operadores pode ser usada para este fim, com exceção dos operadores `.`, `.*`, `::` e `?:`). Este recurso sintático é chamado *sobrecarga de operador*. Um operador sobrecarregado é um método cujo nome é formado pela palavra reservada `operator` seguida do símbolo do operador que se deseja sobrecarregar. Os parâmetros formais do método devem ser compatíveis com o tipo de operador, obrigatoriamente. O valor de retorno do método deve opcionalmente ser compatível com a semântica do operador.

Exemplo 1.5

```
struct Complex
{
    ...
    Complex operator +(const Complex &c) const // add
    {
        return Complex(a + c.a, b + c.b);
    }
    Complex operator -(const Complex &c) const // sub
    {
        return Complex(a - c.a, b - c.b);
    }
};
```

Uma vez sobrecarregados os operadores + e - na classe `Complex` pode-se escrever:

```
Complex c4 = c1.operator +(c2);
Complex c5 = c3 + c4; // Complex c5 = c3.operator +(c4);
```

Na última declaração acima, `c3 + c4` parece ser uma expressão aritmética do tipo adição cujos operandos são dois números complexos. Porém, a expressão é, de fato, um envio da mensagem cujo seletor é `operator +` e cujo argumento é `c4` ao receptor `c3`. Em resposta, o método `Complex::operator +(const Complex &) const` é acoplado e então invocado.

Observações

- Dentro do corpo de um método declarado em uma classe, o receptor da mensagem acoplada ao método é identificado pela palavra reservada `this`, a qual é uma constante que armazena o endereço do receptor da mensagem. Este endereço é implicitamente passado como argumento ao método durante o acoplamento. Então, o método `operator +` da classe `Complex` poderia ser escrito como:

```
struct Complex
{
    ...
    Complex operator +(const Complex &c) const // add
    {
        return Complex(this->a + c.a, this->b + c.b);
    }
    ...
};
```

Observe que o operador + é binário, ou seja, requer dois operandos. No método acima, estes dois operandos são: o objeto `*this`, receptor da mensagem (sendo `this` implicitamente passado como argumento pelo compilador), e o objeto `c` cuja referência deve ser explicitamente passada como argumento na mensagem.

- Em C++, um método `const` é aquele que não altera os atributos do conteúdo de `this`, ou seja, do receptor da mensagem. O modificador `const` faz parte da assinatura do método, ou seja, dois métodos com o mesmo nome simples e mesmo número e tipo de parâmetros, um deles `const` e outro não, são métodos distintos.
-

Exemplo 1.6

Complete a implementação da classe `Complex` com as operações de multiplicação, divisão e conjugado.

```
struct Complex
{
    ...
    Complex& operator +=(const Complex& c)
    {
        a += c.a; // ou this->a += c.a;
        b += c.b; // ou this->b += c.b;
        return *this;
    }
    Complex& operator -=(const Complex &c)
    {
        a -= c.a; // ou this->a -= c.a;
        b -= c.b; // ou this->b -= c.b;
        return *this;
    }
    Complex operator *(const Complex &c) const
    {
        return Complex(a * c.a - b * c.b, a * c.b + b * c.a);
    }

    //
    // TODO: escreva aqui os demais métodos
    //
};
```

Destruição de Objetos

Após o uso objetos são destruídos. A destruição envolve as seguintes operações (inversas das operações realizadas na criação):

1. Finalização do objeto. Efetuada por um método especial declarado na classe do objeto chamado *destrutor*. O código do destrutor deve ser responsável pela "limpeza da casa" (por exemplo, fechamento de arquivos, liberação de memória e dos demais recursos alocados pelo objeto na criação ou ao longo da vida útil, etc.). Só pode haver um destrutor declarado em uma classe. Em C++, o compilador provê um destrutor se um não for declarado, cujo corpo é vazio, ou seja, que não faz nada.
2. Liberação da memória utilizada para armazenamento dos atributos do objeto.

O momento de destruição de um objeto depende de como este foi criado. Objetos criados estaticamente são automaticamente destruídos quando cessa o tempo de vida do escopo no qual foram criados, ou seja, objetos locais são destruídos quando o fluxo de execução é transferido para fora do bloco onde foram declarados, enquanto objetos globais são automaticamente destruídos ao término da execução do programa. Objetos criados dinamicamente têm que ser explicitamente destruídos com o operador `delete`.

Exemplo 1.7

A classe `IntList` declara um método destrutor que esvazia a lista. Em C++ um destrutor não possui parâmetros e tem o nome começado por `~` seguido do nome da classe. Uma implementação do destrutor de `IntList` é

```
inline
IntList::~IntList() // destrutor
{
    clear();
}
```

sendo o método `clear()` escrito como

```
void
IntList::clear()
{
    while (head != 0)
    {
        IntNode* temp = head;

        head = head->next;
        delete temp;
    }
    tail = 0;
    numberOfElements = 0;
}
```

Observações

- O operador **delete** invoca o destrutor declarado na classe do objeto e em seguida libera a memória do heap ocupada pelo objeto:
delete p;
 - Atenção: manipulação explícita de ponteiros é uma das causas mais comuns de erros de programação em C++.
-

Cópia Rasa versus Cópia Profunda

Um construtor de cópia deve implementar a semântica de cópia para os objetos de uma classe. Se nenhum construtor de cópia é declarado, o compilador sempre provê um cuja semântica de cópia é *shallow-copy*, ou *cópia rasa*, isto é, uma cópia membro a membro dos atributos do argumento passado para o construtor para os atributos do objeto sendo inicializado.

Cópia rasa só é distinta de *cópia profunda*, ou *deep-copy*, para objetos representados por ponteiros ou referências. Como exemplo, seja o ponteiro

```
x* p;
```

Uma cópia rasa de `p` é um endereço cujo valor é o mesmo endereço de `p`, ou seja, uma cópia rasa de `p` aponta para o mesmo objeto que `p` aponta. Uma cópia profunda de `p` é um endereço que aponta para uma cópia do objeto para o qual `p` aponta, ou seja, `p` e sua cópia profunda apontam para *dois* objetos iguais. Para exemplificar, seja a função:

```
IntList f()
{
    // A declaração abaixo invoca o construtor default em IntList:
    IntList list;

    ...
    // A sentença abaixo invoca o construtor de cópia e
    // o destrutor de IntList:
```

```

// -----
// IntList temp(list);
// list.~IntList();
// -----
return list;
}

```

A função declara um objeto local chamado `list` que é retornado ao término da função `f()`. Note que a declaração

```
IntList list;
```

corresponde à criação estática de um objeto, ou seja, além da alocação de memória para o armazenamento dos atributos declarados em `IntList`, um método de inicialização do objeto `list` é invocado, neste caso o construtor default. Como o objeto `list` é criado localmente, este será destruído imediatamente antes do fluxo de execução ser transferido para fora do escopo onde foi declarado, o que ocorre com a execução da sentença

```
return list;
```

Isto significa que, ao retornar, a função `f()` destrói `list`, ou seja, invoca o destrutor declarado na classe `IntList`. Este destrutor destrói todos os elementos da lista. A fim de evitar que o valor de retorno seja inútil (uma vez que o objeto local `list` será destruído), uma cópia temporária deste é efetuada, sendo esta cópia retornada pela função `f()`. Este objeto temporário é inicializado via construtor de cópia. Note, porém, que se não houver construtor de cópia em `IntList`, o compilador provê um que implementa a semântica de *shallow-copy*, ou cópia rasa. Dessa maneira, o objeto temporário teria os atributos `head`, `tail` e `numberOfElements` com os mesmos valores dos atributos do objeto `list`. Como `list` será destruído imediatamente após a cópia, o objeto temporário terá os ponteiros `head` e `tail` "pendentes", isto é, apontando para endereços de memória inválidos. Para evitar isso, um construtor de cópia deve ser explicitamente definido em `IntList`, o qual deve implementar a semântica de *deep-copy*, ou cópia profunda. Uma implementação deste construtor seria:

```

IntList::IntList(const IntList& c)
{
    this->head = 0;
    this->tail = 0;
    this->numberOfElements = 0;
    for (IntNode* node = c.head; node != 0; node = node->next)
        this->add(node->value);
}

```

O construtor faz uma cópia (profunda) de todos os elementos de `c`, os quais são adicionados ao objeto sendo construído. Com este construtor de cópia, `f()` funciona (não há ponteiros "pendentes"), mas note que no momento de retorno da função, antes do objeto local `list` ser destruído, existirão *duas* listas em memória, cada uma com seus próprios elementos, mas cujos valores inteiros são iguais, isto é, a lista temporária é uma cópia profunda da lista `list`. Se `list` contiver muitos elementos, em funções similares à `f()` esta duplicação (seguida de destruição) será ineficiente em termos de consumo de memória e tempo de processamento. Uma alternativa para evitar isso é proibir o uso público do construtor de

cópia e, por conseguinte, de funções similares à `f()`, declarando-se o construtor de cópia na seção privada da classe `IntList` (este assunto será abordado mais adiante).

A cópia de um objeto não é feita apenas na inicialização de outro objeto sendo construído, mas também em expressões de atribuição, como no exemplo a seguir:

```
// Cria list1 e list2 com construtor default:
IntList list1, list2;

// Adiciona alguns elementos à list1:
for (int i = 1; i <= 10; i++)
    list1.add(i);

// Cria list3 como uma cópia de list1 (com construtor de cópia):
IntList list3 = list1; // ou IntList list3(list1);

// Copia list3 em list2:
list2 = list3;
```

A última linha acima é uma *expressão de atribuição* na qual `list2` é e-valor e `list3` é d-valor (ambos os objetos foram criados em *declarações* feitas anteriormente). Como não se trata da criação de um objeto, a cópia não é feita através da invocação do construtor de cópia, mas sim da invocação do *operador de atribuição* da classe `IntList`. A expressão, portanto, pode ser escrita como

```
list2.operator =(list3);
```

O operador de atribuição sobrecarregado em uma classe tem como único parâmetro uma referência constante para um objeto da própria classe e retorna uma referência para um objeto da própria classe (geralmente **this*). Se não houver em uma classe a sobrecarga do operador de atribuição, o compilador provê um automaticamente, o qual, tal como feito com o construtor de cópia, implementa a semântica de cópia rasa. Se esta não for adequada, o operador de atribuição deve ser sobrecarregado a fim de implementar a semântica de cópia requerida. Geralmente, se um construtor de cópia for definido em uma classe, então o operador de atribuição também o é. Uma implementação para a classe `IntList` seria:

```
IntList&
IntList::operator =(const IntList& c)
{
    // Deleta os elementos de *this:
    clear();
    // Copia os elementos de c para *this:
    for (IntNode* node = c.head; node != 0; node = node->next)
        this->add(node->value);
    return *this;
}
```

A fim de se evitar os problemas de performance decorrentes da cópia de listas com muitos elementos — conforme discutido na implementação do construtor de cópia de `IntList` —, pode-se declarar também o operador de cópia na seção privada da classe, evitando-se assim o uso público de expressões de atribuição como no exemplo acima.

1.2 Atributos e Métodos de Classe

Ao contrário de *atributo de instância*, um *atributo de classe* não pertence à estrutura de dados de objetos da classe. Há apenas uma cópia do atributo (que pode ser entendido como sendo de propriedade da classe), ao invés de uma cópia para cada objeto da classe. Os objetos da classe compartilham essa cópia única. Em C++, um atributo de classe é definido com o modificador **static**. Um atributo de classe é declarado na classe (geralmente em um arquivo `.h`), mas deve ser definido em um arquivo `.cpp`.

Um *método de classe* não é invocado como resultado do acoplamento mensagem/método. Um método de classe não pode acessar atributos de instância de **this** (não é possível usar **this** em um método de classe, pois não há um objeto receptor de uma mensagem acoplada ao método). Um método de classe se parece como uma função global que "pertence" a uma classe. Em C++, um método de classe é definido com o modificador **static**.

Exemplo 1.8

Classe `Color` em `Figure2D`.

1.3 Tratamento de Exceções

O tratamento de exceções constitui-se em:

1. Lançamento da exceção. Quando um método verifica a ocorrência de uma situação excepcional, este pode "lançar" uma exceção. Lançar uma exceção significa criar um objeto que transportará em seus atributos ou definirá, através de seus métodos (conforme declarados em sua classe), as informações relevantes sobre a exceção detectada. Uma vez criado, este objeto é lançado em uma sentença **throw**. Em C++, uma exceção pode ser um objeto de qualquer tipo. O código que segue uma sentença **throw** não é executado no caso da exceção ser lançada.
2. Tratamento da exceção propriamente dita. Todo código em que uma exceção pode ocorrer deve ser enclausurado em um bloco **try**. Se em algum ponto do **try** uma exceção for lançada, o código restante após o ponto de lançamento não é executado. Após o bloco **try**, deve-se ter um ou mais blocos **catch**, cada um deles parametrizado com um dos tipos de exceções que podem ocorrer no **try**. Na ocorrência de uma exceção, o fluxo de controle é transferido para o primeiro bloco **catch** cujo parâmetro é de um tipo tal que o tipo da exceção lançada é igual ou pode ser convertido para o tipo do parâmetro. O código do **catch** efetua o tratamento da exceção e o fluxo de controle é desviado para o código após o último **catch** (a não ser que outra exceção seja lançada de dentro do **catch**). Se não houver um **catch** para o tipo da exceção lançada, uma função global de tratamento de exceções é invocada (geralmente, esta aborta a execução do programa).

Exemplo 1.9

A classe `my_out_of_memory` representa uma exceção a ser lançada se a função `my_alloc()` não for capaz de alocar um bloco de memória do tamanho requisitado. A captura e tratamento da exceção, se lançada em `my_alloc()`, é feita na função de teste `q()`.

```

#include <stdlib.h>

struct my_out_of_memory
{
    my_out_of_memory(size_t size)
    {
        this->size = size;
    }

    size_t size;
};

void* my_alloc(size_t size)
{
    void *ptr = malloc(size);

    if (ptr == 0)
        throw my_out_of_memory(size);
    return ptr;
}

void q()
{
    void* b1 = 0;
    void* b2 = 0;
    void* b3 = 0;

    try
    {
        b1 = my_alloc(1000);
        b2 = my_alloc(3000);
        b3 = my_alloc(4000);
        // faça algo
        ...
    }
    catch (my_out_of_memory& e)
    {
        printf("out of memory (%d bytes)\n", e.size);
    }
    free(b1);
    free(b2);
    free(b3);
}

```

Capítulo 2

Propriedades da Orientação a Objetos

2.1 Encapsulamento

Encapsular significa esconder, na definição da classe, os membros que representam a estrutura e o comportamento interno dos objetos da classe. Geralmente, são escondidos todos os atributos, enquanto os métodos são tornados públicos. Entretanto, podem-se esconder também métodos, bem como tornar públicos alguns atributos. Tudo depende do projeto e do bom senso.

"Esconder um membro" em C++ significa declará-lo na seção *privada* (definida pela palavra reservada `private`) ou *protegida* (definida pela palavra reservada `protected`) da classe. Um membro privado só pode ser acessado em métodos da classe onde foi declarado. Um membro protegido pode ser acessado por métodos da própria classe e de classes derivadas publicamente, direta ou indiretamente, da classe onde foi declarado. Membros públicos, ou seja, declarados na seção *pública* (definida pela palavra reservada `public`) da classe, podem ser acessados por quaisquer métodos e definem a interface da classe.

Para um atributo privado ou protegido, o acesso a este só pode ser efetuado por métodos quaisquer de forma indireta, através de métodos declarados na classe do atributo chamados *setters* e *getters*. Um *setter* é utilizado para atribuição de um novo valor para um atributo, enquanto um *getter* é utilizado para obtenção do valor do atributo. Um atributo para o qual há um *getter* e/ou um *setter* é uma propriedade do objeto. Embora aconselhável esconder um atributo, seu acesso por *getters* ou *setters*, além do acréscimo de métodos na interface da classe, pode adicionar ineficiência a um programa. Em C++, tal ineficiência é evitada com o uso de métodos *inline*. Em uma mensagem na qual um método *inline* é acoplado, não há o *overhead* de uma chamada de função; ao invés disso, o corpo do método é expandido "em linha" no local da mensagem. Com isso, o código gerado fica maior, dependendo do tamanho do corpo do método, como visto anteriormente.

As vantagens do encapsulamento são:

- Melhor organização do código.
- Segurança: atributos encapsulados não podem ter seus valores inadvertida ou maliciosamente modificados por métodos que não têm acesso aos atributos.
- Manutenibilidade: alterações na estrutura de dados e em métodos encapsulados de uma classe não são refletidas nos métodos de outros objetos usuários da classe (a não ser que essas alterações modifiquem a interface da classe).

Os *getters* e/ou *setters* públicos declarados em uma classe permitem que membros privados ou protegidos de objetos da classe possam ser acessados por quaisquer funções ou métodos de um programa. Há situações, contudo, em que certos membros privados ou protegidos de objetos de uma classe A devem ser acessados não por métodos quaisquer, mas somente por métodos acoplados a mensagens cujos receptores são objetos de uma classe B. Casos assim podem ocorrer quando há um relacionamento forte entre objetos das classes A e B. Não seria adequada a declaração de *getters* e/ou *setters* públicos para manipulação dos membros

escondidos da classe A, pois isto permitiria um acesso que deveria ser exclusivo a objetos da classe B. Nestas circunstâncias, a classe A pode declarar que a classe B é sua *amiga*, o que significa que métodos executados por objetos da segunda podem acessar membros de objetos da primeira.

Exemplo 2.1

No código fonte no Moodle, a classe `Shape2D` declara que a classe `Figure2D` é “amiga”:

```
class Shape2D
{
    friend class Figure2D;
    Shape2D* next;
    ...
};
```

Enquanto a declaração fere a propriedade do encapsulamento (e, portanto, deve ser feita criteriosamente), seu uso se justifica, neste caso, porque há uma relação forte de dependência entre as classes, visto que um objeto do tipo `Figure2D` é um contêiner de objetos de tipo `Shape2D`. Assim, não é necessária a declaração de um *getter* e um *setter* em `Shape2D` a fim de que `Figure2D` possa manipular o atributo privado `Shape2D::next` (aliás, a declaração de tal *getter* e *setter* em `Shape2D` permitiria que qualquer outro objeto ou função pudesse manipular o atributo, o que não deve ser permitido, visto que essa é uma exclusividade de `Figure2D`).

2.2 Herança

Herança é o mecanismo através do qual um objeto de uma classe *y* *herda* todos os membros de uma classe *x*. A classe *x* é chamada *superclasse* (ou *classe base*, em C++) e a classe *y* é chamada *subclasse* (ou *classe derivada* em C++). Uma classe base representa uma *generalização*; uma classe derivada representa uma *especialização*. C++ admite herança múltipla, i.e., uma classe pode ser derivada de duas ou mais classes base.

Há várias maneiras de uma classe derivada especializar uma classe base:

- Adicionando novos métodos e/ou atributos;
- Redefinindo a implementação de métodos da classe base;
- Não fazendo nada (a própria declaração de um tipo novo é significativa em si).

O grande benefício da herança é a reutilização de código, o que evita a replicação e pode, mas nem sempre, facilitar a manutenção, desde que alterações ou extensões em uma classe base sejam feitas somente nas partes privadas da classe.

Exemplo 2.2

Veja, no Moodle, as classes de formas 2D derivadas de `Shape2D`.

2.3 Polimorfismo

Polimorfismo é a capacidade de operadores ou métodos com o mesmo nome implementar várias operações distintas. Em C++, por exemplo, o operador `+` é usado para a adição de dois inteiros, de dois reais, de um real e um inteiro, etc. Em Java, o operador também é usado para concatenação de strings. Além disso, em C++, é possível a sobrecarga de operadores, ou seja, a definição de funções cujo nome simples é dado pela palavra reservada **operator** seguida do símbolo do operador (os argumentos da função devem ser compatíveis com o número de argumentos exigidos pelo operador), conforme visto anteriormente. Assim, pode-se escrever, para uma classe `x` qualquer:

```
X operator +(const X&, const X&);
```

Com isso, o operador `+` passa a ser usado também para a adição de dois objetos da classe `x`:

```
X x1;
X x2;
X x3;
...
x3 = x1 + x2; // ou x3 = operator +(x1, x2);
```

O operador também pode ser declarado como membro da classe:

```
class X
{
public:
    ...
    X operator +(const X&) const;
    ...
};
```

Assim:

```
x3 = x1 + x2; // ou x3 = x1.operator +(x2);
```

No caso de polimorfismo envolvendo funções que não sejam operadores, podem-se ter funções ou métodos com o mesmo identificador, mas que são distinguidos pelo número e/ou tipo dos parâmetros. Como exemplo, sejam os métodos da classe `A`:

```
class A
{
    int a;
public:
    virtual void f();
    void f(int);
    virtual void f(int, float);
    void f(float, int);
};
```

Há quatro versões do método `f()` na classe. No exemplo a seguir, os métodos acoplados às respectivas mensagens são determinados em tempo de compilação em função do número e/ou tipo dos argumentos passados à mensagem. Quando o acoplamento mensagem/método é resolvido em tempo de compilação, é chamado *acoplamento estático* ou *junção anterior*.

```
A a;
a.f(); // A::f()
a.f(2); // A::f(int)
```

Na classe `A` há quatro versões declaradas do método cujo nome simples é `f`, ou seja, uma instância de `A` pode responder a quatro tipos distintos de mensagens cujo seletor é `f`. Em casos como o exemplo acima, onde são declarados na *mesma* classe métodos com o *mesmo* nome simples mas com número e/ou tipos *distintos* de parâmetros, diz-se que tais métodos são *sobrecarregados*. Portanto, cada método declarado em `A` com o nome simples `f` é um método sobrecarregado. O mesmo vale para operadores e funções declaradas em mesmo escopo. Há, contudo, uma outra faceta de polimorfismo, relacionada com o mecanismo de herança. Uma classe derivada pode *redefinir* um método de uma classe base, com o mesmo identificador, mesmo número e tipos de parâmetros, mesmo tipo de retorno mas com um corpo que pode ser diferente. Isto é feito na classe `B` a seguir, derivada de `A`:

```
class B: public A
{
    int b;
    int c;

public:
    virtual void k();
    void f();
    void f(int);
};
```

Note que uma instância de `B` pode responder a seis mensagens cujo seletor é `f`: quatro para métodos herdados de `A` e duas para métodos declarados em `B`. Na classe `B` há sobrecarga de métodos (métodos na mesma classe com mesmo nome mas parâmetros diferentes) e, além disso, redefinição de dois métodos herdados da classe base `A`.

Em C++, para mensagens enviadas a objetos criados estaticamente, o acoplamento sempre será estático, como no exemplo abaixo:

```
B b;
b.f(); // B::f()
b.f(2); // B::f(int)
b.A::f() // A::f()
```

As coisas mudam quando uma mensagem é enviada a um objeto através de um ponteiro (ou referência). Em C++, um ponteiro (ou referência) para uma classe base pode receber o endereço (referência) de um objeto desta classe ou de qualquer classe que dela deriva, direta ou indiretamente. Assim, no código abaixo:

```
A* pa;
...
pa->f();
```

pode-se perguntar qual método será acoplado à mensagem, uma vez que o ponteiro `pa` pode receber o endereço de um objeto da classe `A`, da classe `B` ou de qualquer outra que derive de `A`. A resposta a essa questão depende do método `A::f()` ser *virtual* ou não. Se o método

não for virtual, então `A::f()` será acoplado (independentemente do tipo do objeto cujo endereço é atribuído a `pa`), uma vez que `pa` é um ponteiro para a classe `A`, ou seja, o acoplamento é estático (determinado em tempo de compilação). Se o método for virtual, então o método a ser acoplado à mensagem será aquele da classe do objeto cujo endereço, em tempo de execução, for atribuído ao ponteiro `pa`. Esse tipo de acoplamento é chamado *acoplamento dinâmico* ou *junção posterior*, pois é resolvido em tempo de execução.

Em uma classe derivada, a redefinição de um método não virtual herdado de uma classe base *esconde* o método da classe base. A redefinição de um método virtual em uma classe derivada *sobrepõe* o método da base. Note a diferença: métodos com o *mesmo* nome mas parâmetros *diferentes* declarados na *mesma* classe são métodos *sobrecarregados*, enquanto que métodos declarados em uma classe derivada com o *mesmo* nome, *mesmos* parâmetros e *mesmo* tipo de retorno de métodos herdados de uma classe base são métodos *redefinidos*. Um método redefinido *sobrepõe* ou *esconde* o método da base, caso este seja *virtual* ou não, respectivamente. Métodos virtuais permitem acoplamento dinâmico.

Acoplamento dinâmico tem um custo em termos de eficiência. Em C++, a junção posterior é implementada através de *tabelas de ponteiros de métodos virtuais*. Toda classe que tiver declarado pelo menos um método virtual terá associada uma tabela cujas entradas serão os endereços do código de cada um dos métodos virtuais declarados na classe. Todo método virtual possui como atributo um índice para a entrada correspondente na tabela de ponteiros de métodos virtuais (TPMV) de sua classe. A TPMV de uma classe derivada copia a TPMV da classe base, ajustando as entradas para apontar para aqueles métodos que foram sobrepostos e adicionando novas entradas se novos métodos virtuais forem declarados na classe derivada.

Por exemplo, a TPMV da classe `A` é:

0	<code>&A::f()</code>
1	<code>&A::f(int, float)</code>

A TPMV da classe `B` é:

0	<code>&B::f()</code>
1	<code>&A::f(int, float)</code>
2	<code>&B::k()</code>

Uma instância de uma classe com métodos virtuais possui em sua estrutura, além dos atributos declarados na classe, um ponteiro para a TPMV da classe (espaço para este ponteiro é alocado pelo compilador como o primeiro atributo do objeto). O acoplamento do método (virtual) à mensagem `pa->f()`, em C++, é feito da seguinte forma:

1. O endereço da TPMV do objeto apontado por `pa` é obtido (é o primeiro atributo do objeto);
2. O endereço do método a ser acoplado é obtido da TPMV e corresponde à entrada do índice do método virtual `f()` (neste caso, zero);

3. Os argumentos do método e `this` são empilhados e o método é então invocado.

O grande benefício da junção posterior é código novo funcionando com código velho, eliminando construções do tipo switch-case.

Observações

- Um *método abstrato*, em C++, é um método *virtual puro*. Toda classe que declarar pelo menos um método abstrato é uma *classe abstrata*. Uma classe abstrata não admite instância.
- Uma classe derivada de uma classe base abstrata não será abstrata se sobrepor todos os métodos abstratos herdados da base (e não declarar novos métodos abstratos).

Exemplo 2.3

Método `Figure2D::print()` em `Figure2D` no Moodle.

À mensagem `shape->print()` será acoplado o método correto (de `Line2D`, `Circle2D` ou `Rectangle2D`) e o código irá funcionar para qualquer objeto cuja classe derivar de `Shape2D` e sobrepor o método abstrato `Shape2D::print()`.

Capítulo 3

Classes Paramétricas em C++

C++ implementa *templates*, os quais podem ser usados para definição de classes e funções. Para exemplificar seu uso em classes, sejam as classes `IntNode` e `IntList` (introduzida no Capítulo 1) abaixo, as quais representam um elemento de uma lista linearmente encadeada de números inteiros e a lista propriamente dita:

```
struct IntNode
{
    int value;
    IntNode* next;
};

class IntList
{
    IntNode* head;
    IntNode* tail;
    int numberOfElements;

public:
    IntList(); // construtor
    ~IntList(); // destrutor
    void add(int);
    bool remove(int);
    void clear();
    bool contains(int) const;
    void iterate(void (*)(int)) const;
};
```

Vamos supor agora que se queira implementar uma lista de outro tipo qualquer, por exemplo, `float` ou `Complex`. Sem *templates* há duas alternativas:

1. Replicar a implementação da lista, criando novas classes `FloatNode` e `FloatList` (ou `ComplexNode` e `ComplexList`) iguais a `IntNode` e `IntList`, respectivamente, mas trocando o tipo `int` por `float` (ou `Complex`). Isso é indesejável por razões óbvias, dentre as quais a necessidade de alterações manuais no código replicado sempre que um novo método for adicionado à classe ou que algum método já existente for alterado.
2. Para evitar a replicação, definir uma classe única de lista de objetos quaisquer:

```
class List
{
    ListElement* head;
    ListElement* tail;
    int numberOfElements;

public:
    List(); // construtor
    ~List(); // destrutor
    void add(Object*);
    bool remove(Object*);
    void clear();
    bool contains(Object*) const;
    void iterate(void (*)(Object*)) const;
};
```

Nesta alternativa, um elemento de uma lista é um objeto da classe `ListElement`:

```
struct ListElement
{
    Object* value;
    ListElement *next;
};
```

A classe abstrata `Object` deve ser a classe base de todo objeto que pode ser valor de um elemento de uma lista. Uma interface mínima para a classe poderia ser:

```
class Object
{
public:
    virtual ~Object() {} // destrutor
    virtual Object* clone() const = 0;
    virtual bool equals(const Object&) const = 0;
    virtual String toString() const = 0;
};
```

Assim, para adicionar números inteiros a esta lista de objetos, seria preciso criar uma classe derivada de `Object` para representar um objeto do tipo inteiro (pois a lista genérica, agora, é uma lista de objetos, e o tipo primitivo `int` não deriva de `Object`):

```
class Int: public Object
{
public:
    Int() {}
    Int(int value) { this->value = value; }
    operator int() const
    {
        return value;
    }
    Int& operator =(int value)
    {
        this->value = value;
        return *this;
    }
    Object* clone() const;
    bool equals(const Object&) const;
    String toString() const;

private:
    int value;
};
```

Da mesma forma, para adicionar números complexos a um objeto do tipo `List`, a classe `Complex` (introduzida no Capítulo 1) deveria agora ser derivada de `Object` e sobrepôr os métodos abstratos dela herdados.

A vantagem desta abordagem é que agora há somente uma classe de lista linear de objetos. Qualquer extensão ou alteração é efetuada apenas na classe `List`. O inconveniente é que um objeto do tipo `List` seria um contêiner de quaisquer objetos cujo tipo fosse derivado direta ou indiretamente da classe `Object`, ou seja, pode-se ter na mesma lista elementos

cujo valor é um objeto `Int` e elementos cujo valor é um objeto `Complex`. Às vezes é isso mesmo que se quer, mas nem sempre. Além disso, se um tipo primitivo quiser ser admitido em uma lista, é necessário criar uma classe que represente este tipo primitivo, como feito com números inteiros (classe `Int`). Por fim, classes que em outras circunstâncias não precisariam derivar de `Object`, agora precisam, como é o caso de `Complex`. Isso pode ser evitado em C++ com o uso de *templates*.

Um *template* de uma classe, ou *classe paramétrica*, não é de fato uma classe, mas um “molde” a partir do qual o compilador C++ gera uma classe. Para exemplificar, sejam os seguintes *templates* de elementos e de lista linear cujos valores são de um tipo `T` qualquer:

```
template <typename T>
struct ListElement
{
    T value;
    ListElement<T>* next;
    // Construtor
    ListElement(T value) { this->value = value; }
};

template <typename T>
class List
{
    ListElement<T>* head;
    ListElement<T>* tail;
    int numberOfElements;

public:
    List(); // construtor
    ~List(); // destrutor
    void add(T);
    bool remove(T);
    void clear();
    bool contains(T) const;
    void iterate(void (*)(int)) const;
};
```

No exemplo acima, `typename T` é o parâmetro do *template*, o qual representa um tipo qualquer chamado `T`. Note, no corpo da classe paramétrica, que `T` é usado para definir os tipos de atributos, valores de retorno de métodos e parâmetros de métodos da classe. Um *template* pode ter qualquer número positivo de parâmetros.

Definido um *template*, pode-se gerar uma classe através dele. Gerar uma classe a partir de um *template* significa *instanciar o template*. Para tal, basta escrever o nome da classe paramétrica e fornecer um argumento para cada um dos parâmetros do *template*. Para se gerar uma classe de lista de números inteiros, então, escreve-se `List<int>`. Supõe-se que um *template* seja instanciado somente uma vez pelo compilador; Quando um *template* é *instanciado*, todo o código definido no molde é gerado, trocando-se o parâmetro pelo argumento (no exemplo, todo `T` é trocado por `int`). Observe a declaração:

```
List<Complex> cl;
ou, alternativamente
```

```
typedef List<Complex> ComplexList;
ComplexList cl;
```

Nas declarações acima ocorre a geração da classe que representa uma lista de números complexos, se esta ainda não tiver sido gerada pelo compilador (instanciação do *template*), e em seguida a criação de um objeto chamado `cl` do tipo lista de números complexos (instanciação da classe).

Exemplo 3.1

Estude a classe paramétrica `List<T>` no Moodle. Note que, para que o *template* possa ser corretamente instanciado pelo compilador, devem ser definidas sobre o tipo `T` as operações de cópia e comparação (identifique as partes do código que evidencia isso).

Cada instanciação de um *template* com diferentes argumentos *replica* todo o código definido no molde, com a vantagem de ser executada automaticamente pelo compilador. Mas tenha em mente que a replicação ocorre. Portanto, embora seja a solução interessante em muitos casos, *templates* devem ser usados criteriosamente. No caso de lista, os métodos são simples e suas replicações não aumentariam demais o tamanho do programa objeto. Isto não ocorre, contudo, em estruturas que requerem um código mais complexo, por exemplo árvores AVL. A solução, neste caso, é definir uma classe-base que implementa os métodos mais complicados e que operam sobre um tipo genérico, como exemplificado a seguir:

```
class AVLTreeBase
{
private:
    AVLNode* root;
    int numberOfNodes;

public:
    class Node
    {
        Node* rgt;
        Node* lft;
        int balance;
        friend class AVLTreeBase;
    }

    void add(Node*);
    ... // demais métodos
};
```

A classe `AVLTreeBase` implementa as operações de inserção, remoção e atravessamento (iteração) de uma AVL, considerando que os elementos são do tipo `AVLTreeBase::Node`. Estes métodos são declarados protegidos em `AVLTreeBase`. Definida a classe base, faz-se:

```
template <typename T>
struct AVLNode: public AVLTreeBase::Node
{
    T value;
    AVLNode(T value) { this->value = value; }
};
template <typename T>
```

```

class AVLTree: public AVLTreeBase
{
public:
    void add(T value)
    {
        AVLTreeBase::add(new AVLNode<T>(value));
    }
    ... // demais métodos
};

```

O template `AVLNode<T>` gera classes cujos objetos são nós de uma AVL com valor do tipo `T`, enquanto o template `AVLTree<T>` gera classes cujos objetos são AVLs de elementos do tipo `T`. Note que a instânciação deste template não replica o código complicado da estrutura de dados AVL, o qual foi implementado em `AVLTreeBase`.